Kutty Banerjee

6071 Raleigh Road San Jose CA 95123 kutty.banerjee@gmail.com 925-209-7204

SUMMARY

Proficient in Software-Hardware interfaces, CPU-GPU Optimizations, Low Latency Data Flow, Parallel Computing and Compiler Runtimes. Have Implemented and Lead Engineering Projects in Parallel Computing and Low Latency Runtimes.

EXPERIENCE

Principal Engineer, Apple

Cupertino, CA — Aug 2013 - Present

Engineering Lead for Overall Latency and QoS for the GPU Stack on iPhone, iPad, AppleTV, Watch and future unannounced Latency Sensitive Products. This involves Driving changes in upcoming Hardware, CPU optimizations at Driver/Firmware layers, Coordinating across various teams, Driving test plans and reporting Weekly Status to Sr Director. Filed 10 Apple patents in this area.

CPU Optimizations for Graphics Firmware

- iOS11 Optimizations: Reduced Firmware Interrupt latency on ARM CPU for kicking work to GPU by ~4x. On iPhone7 **reduced latency from 21us to 6us**. On iPhone 8/X **reduced latency from 12us to 2.8us**. This latency contributes towards frame time with around 200 commands per frame in modern rendering workloads operating at 120Hz.
- The Optimizations include Data Packing to fit adjacent Cache lines, moving cold code out of hot icache path, fast path early detection and branch elision, 2 Level Manual Data Prefetching, NEON writes for bulk register updates.
- Also worked with LLVM backend teams to improve Compiler Instruction Scheduling logic for the CPU target.
- Worked with CPU Hardware team to design next generation Power Efficient CPU for Graphics Firmware. Responsible for Modelling futiristic Graphics Workloads, measuring IPC/CPI for critical stall paths and improving memory fetch latencies with techniques like "Runahead Execution", better Prefetchers, better Branch Predictors.
- Acted as the point of contact between CPU and GPU HW teams.

Graph Based Command Execution

- Proposed, designed and lead a team of engineers to convert streaming Graphics commands into a Data Dependency Graph at runtime. This allows **O(1) sub-10us worst case** command selection logic for the case where the Interrupt handler does not find a ready Command to feed to the GPU. Used to be ~500us in the previous implementation.
- Nodes represent stages of a Pipeline executed by the Firmware, edges represent Data Dependencies across nodes. Event Processing can now be done by following high priority edges instead of testing to see which command is ready for execution.
- This also allows Parallelizing independent commands into separate Tiling/Rendering Hardware Units. Also minimizes Pipeline Fence stalls by queuing independent commands to be in flight in HW simultaneously. Graph allows Looping subgraphs to allow reuse of submitted sub graphs (**Primary inventor and Patent Filed**)

GPU Fair Scheduler / Quality of Service

- Implemented a Prioritized Fair Scheduler that balances GPU utilization evenly based on Priority Level.
- Released as part of iOS9 to support "**Split Screen Multitasking on iPads**". Allows multiple applications to render simultaneously with the Scheduler providing fairness and starvation freedom across Display Refresh Rate Deadlines (**Patent US9747659**)
- iOS10 added Face Recognition using Deep Learning Inference on the GPU. Changed Scheduler to use Idle cycles for this (when UI is not rendering) to provide smooth UI Experience while staying within the Power Budget. Augmented Scheduler to avoid overlap between Deep Learning tasks and UI Rendering tasks, implemented an early UI inflight detect and preempt policy along with hysteris to keep GPU idle for potentially incoming UI work before using idle cycles for Deep Learning Tasks. (**Patent filed**)
- Worked with Hardware teams to design the Apple GPU QoS System to allow multiple applications in GPU simultaneously and maintaining fair share usage of Shader Cores and Memory Bandwidth between them. This was first released on iPhone 8 and X. Have since proposed and patented several improvements in this area for upcoming undisclosed prodcuts (**Several Patents filed**)
- Worked with HW teams across several GPU generations to bring down Context Switch latency for Geometry, Pixel and Compute tasks.

Latency Analyzing Tools

- Lead development of **AGXAnalyzer**, a Time Series Database that mines traces and plots Critical Path Latencies on the Graphics Stack on any iOS platform.
- Worked with HW teams to develop **Cyclops.** Analyzes CPU simulation runs and plots out critical path stalls / resource contentions in Front End, Back End, Register File and ALU Utilization.

Software Architect, NVIDIA

Santa Clara, CA — May 2005-May 2013

Lead the Kernel development for Fermi GPU Generation from inception to delivery. Implemented Parallel Runtimes, worked on the CUDA Driver Stack. Was part of NVIDIA's first native ARMv8 64-bit CPU Implementation called Denver.

Khronos - Auto Parallelizing Query Processor

- Its a Stack Based Virtual Machine with its own ISA. Instructions are similar to OpenCL kernels and Operands are Memory Tiles
- Instruction Parallelism is obtained by doing Data Flow Analysis of Tile Operands. The resultant Anti Dependency Graph is walked in parallel using Multiple CPU Threads or GPU Work Groups
- Added a Data Aware Scheduler to factor in memory traffic induced by migrating work to coprocessors.
- Supports Branches via Predication and has first class instructions for loops. Supports Software Pipelining for Loop Optimizations.
- Supports Tile renaming to break WAR dependencies both in and outside loops
- Has native builtin Parallel Reduction Operator for Data Parallelism
- Tiles can be split into SubTiles for Data Parallel Operations
- All operators operate on in-memory data structures. Operators can stream memory in from external source before operations and pipeline Computation with I/O Streaming.

Concurrency and Parallel Programming Tools

- Implemented variants of Biased Lock, Ticket Lock, improved SEQLOCK, Multi Writer-Single Reader Lock. The focus being on reducing CAS/fence usage, minimizing cachline traffice and improving lock fairness
- Implemented a Reader-Writer lock where reader does not use an atomic instruction or fence to acquire the reader lock.
- Mailbox based Thread Pool Implementation. Uses dedicated mail boxes each with separate locks to minimize lock contention. Used by Khronos for Work Stealing

ARMv8 Denver CPU Runtime Compiler

- Was part of the Just-in-Time compiler runtime team that did Dynamic Binary Translation of ARMv8 to a native VLIW ISA to extract Instruction Level Parallelism and Memory Level Parallelism. Implemented the following
- Runtime profiled and replaced branches with trap-on-branch instructions to help compact trace scheduling
- Added rollback for this optimization if traps were taken frequently in a small windows of time.
- Implemented Branch Reprofiling to detect dynamic changes in branch weights of a running program and trigger recompilation. Improved libquantum SPECFP by 45%
- Elide Memory fence instructions. This allowed more aggressive instruction scheduling. Transaction rollback is done if a concurrent commit collides with the speculative lines. Helped JVM workloads.
- Did fullstack Linux optimizations for Denver. Implemented ARMv7 Large Page Support FreeBSD Super Pages style.
- Added a new Thread Local Memory type with support in madvise and mmap system calls. Memory Descriptors marked Thread Local are treated separately in the Kernel MM. TLB invalidates, IPI broadcasts and cache snooping for these ranges can be elided on processors where doing so helps performance. Useful for Javascript Thread Local Heaps.

Project Orion - CUDA Cache Coherent CPU/GPU SoC

- Prototyped ARM64 Linux kernel changes to implement Uniform Address Space across CPU and GPUs using Shadow Page Tables. Worked with HW teams to support Page Fault and Fault Aggregation on the GPU. Project was later cancelled, though a lot of the above work was carried forward to the subsequent UVM implementation.

Fermi Generation GPU Kernel Lead

- As the Kernel Driver lead, lead a team of 25 engineers in the Resource Manager Team and oversaw development across Simulation, RTL, Emulation, Bringup and Product Release.
- Role included Task Assignment, Initial Triage, Interaction with Hardware and other Software teams, mentoring new Engineers and Code Review. Responsible for overall progress of the Kernel team for Fermi.
- Was the first engineer to start work on the Kernel/Firmware for Fermi. Wrote and maintained large parts of the kernel code including Thread Scheduling, Interrupt Handling, Memory Management, Page Table Setup, Context Switch Support using

Firmware, Cache mgmt., TLB invalidate batching.

Fermi Command Buffer Scheduling Lead

- Implemented Command Buffer Scheduler in the Driver. Fermi added Hardware support for DX11 Per Node Command Buffers. Replaced the old Time Slice Based Scheduler with Per Node Queues.
- Implemented a Priority based Timeslice mechanism.
- Implemented a Non Stalling Lock Free Interrupt Tree for routing DXG kernel notifiers without delay

EDUCATION

MS Computer Science, Worcester Polytechnic Institute

Worcester, MA — Aug 2003 - May 2005

Thesis: Remote Execution of 3D Graphics for Mobile Systems. Recipient of WPI Institute Fellowship Award for outstanding research work. Publications in Journals and Conferences

BE Computer Science, University of Mumbai

Mumbai, India — July 1999 - May 2003

Publications

1. A Middleware Architecture for Mobile 3D Graphics
   Emmanuel Agu, Kutty Banerjee, Shirish Nilekar, Oleg Rekutin, Diane Kramer
   International Journal of Parallel, Emergent and Distributed Systems Volume 21, Issue 3, 2006
2. Estimating Mobile Memory Requirements and Rendering Time for Remote Execution of the Graphics Pipeline
   Kutty Banerjee, Fan Wu, Emmanuel Agu
   Proceedings of Eurographics 2005
3. Remote Execution for 3D Graphics on Mobile Devices
   Kutty Banerjee, Emmanuel Agu
   International Conference on Wireless Networks, Communications and Mobile Computing, 2005
4. PowerSpy: Fine-Grained Software Energy Profiling for Mobile Devices
   Kutty Banerjee, Emmanuel Agu
   International Conference on Wireless Networks, Communications and Mobile Computing, 2005
5. A Middleware Architecture for Mobile 3D Graphics
   Emmanuel Agu, Kutty Banerjee, Shirish Nilekar, Oleg Rekutin, Diane Kramer
   Third International Workshop on Mobile Distributed Computing, 2006

Patents

1. US20160358305: Starvation free scheduling of prioritized workloads on the GPU
   Kutty Banerjee, Eric O. Sunalp, Tatsuya Iwamoto

(9 other Patents filed in the Pipeline)